

Informe final - FJSSP Rush Order

Scheduling dinámico de un Flexible Job Shop ante un Rush Order en un entorno Digital Twin

Curso: Gestión de Operaciones — Trabajo final **Caso:** perturbación *Rush Order* en $t^* = C_{\max_0}/2$ **Fecha:** julio 2026

1. Introducción

En manufactura personalizada, el programa de producción rara vez se ejecuta tal como se planificó: fallas de máquinas, atrasos y llegada de pedidos urgentes obligan a reprogramar en línea. El paradigma **Digital Twin (DT)** aborda este problema manteniendo una réplica digital del taller sincronizada con el estado físico, de modo que las perturbaciones se detectan al instante y disparan mecanismos de re-optimización (Liu et al., 2022).

Este trabajo aborda el **Flexible Job-Shop Scheduling Problem (FJSSP)** del caso de estudio de Liu et al. (2022) — un taller de autos personalizados con 8 trabajos, 8 máquinas y 5 operaciones por trabajo — sometido a la perturbación **Rush Order**: la llegada de un pedido urgente en el instante $t^* = C_{\max_0}/2$, donde $C_{\max_0}$ es el makespan del programa inicial.

Objetivo: diseñar e implementar una metodología que (i) resuelva el scheduling inicial, (ii) simule la llegada del rush order, (iii) recupere el programa reprogramando solo las operaciones pendientes junto al pedido urgente, y (iv) incorpore un modelo inteligente (XGBoost) que seleccione la estrategia de rescheduling de menor impacto, evaluando el desempeño con métricas cuantitativas (makespan, tiempo de término del rush, *nervousness*, tiempo computacional y porcentaje de recuperación).

2. Marco teórico

FJSSP. Extiende el job shop clásico permitiendo que cada operación se procese en un subconjunto de máquinas factibles, con tiempo dependiente de la máquina. Es NP-duro incluso en instancias pequeñas, por lo que en la práctica se resuelve con metaheurísticas (Liu et al., 2022).

Digital Twin para monitoreo y decisión. El DT mantiene el estado del taller (avance de operaciones, disponibilidad de máquinas, cola de pedidos) sincronizado con el sistema físico. Ante un evento dinámico — aquí, la inserción de una nueva orden en el sistema de pedidos — el DT lo detecta, actualiza el estado y dispara el rescheduling (Liu et al., 2022; Wang et al., 2022).

Algoritmos genéticos para FJSSP. La codificación estándar de dos segmentos MS+OS (*Machine Selection + Operation Sequence*) representa simultáneamente la asignación de máquinas y el orden de las operaciones; con operadores que preservan factibilidad (crossover POX, mutación de reasignación e intercambio) todo cromosoma decodifica a un schedule factible.

Rush order insertion. Cuando llega un pedido urgente, las operaciones ya terminadas se congelan, las que están en proceso no se interrumpen, y las pendientes se reprograman junto con las del pedido urgente (Wang et al., 2022).

Estabilidad y nervousness. Reprogramar todo desde cero maximiza el desempeño teórico pero desestabiliza el taller (cambios de asignación, re-secuenciamientos, replaneación de materiales). Moratori et al. (2010) muestran que estrategias tipo *match-up* que restringen los cambios a una ventana parcial logran calidad comparable con mucho menor perturbación; ello motiva medir la **nervousness** N del programa y usarla como criterio de decisión.

Modelo inteligente. Se usa **XGBoost** (gradient boosting de árboles) como **selector de estrategia de rescheduling**: dado el estado del taller en t^* , predice qué estrategia minimizará una función de impacto Z que combina makespan, término del rush y nervousness. Se eligió por su buen desempeño en datos tabulares pequeños, su interpretabilidad vía importancia de variables y su costo de inferencia despreciable (compatible con decisión en tiempo real).

3. Descripción del problema

Taller de ensamblaje de autos personalizados (laboratorio de Liu et al., 2022):

- **Trabajos:** 8 modelos iniciales (J1–J8); cada uno con **5 operaciones** en secuencia fija: chasis, marco, puerta izquierda, puerta derecha y cubierta frontal.
- **Máquinas:** 8 brazos robóticos (M1–M8). Cada operación tiene un subconjunto de máquinas factibles con tiempos distintos (Tablas 5 y 6 del paper; transcritas en `data/datos_base_extraidos.csv`).
- **Restricciones:** precedencia tecnológica dentro de cada trabajo; cada máquina procesa una operación a la vez; sin interrupción (*non-preemption*).
- **Métrica principal:** makespan (C_{max}).
- **Perturbación:** en $t^* = C_{max}/2$ llega un rush order J9 (reedición del modelo 8, igual que el evento dinámico reportado en el paper base) que debe integrarse al programa en curso.

4. Modelo matemático

4.1 Modelo base (FJSSP, Liu et al. 2022)

Conjuntos: trabajos $J^0 = \{1, \dots, 8\}$, máquinas $M = \{1, \dots, 8\}$, operaciones $O_j = \{1, \dots, 5\}$, máquinas factibles $A_{jh} \subseteq M$. Parámetros: tiempos p_{ijh} . Variables: asignación $x_{ijh} \in \{0, 1\}$, inicios s_{jh} , términos c_{jh} , orden en máquina y_{ijhkl} , makespan $Cmax$.

$$\min Cmax$$

sujeto a: asignación única $\sum_{i \in A_{jh}} x_{ijh} = 1$; definición de término $c_{jh} = s_{jh} + \sum_i p_{ijh} x_{ijh}$; precedencia $c_{jh} \leq s_{j, h+1}$; no solapamiento en cada máquina (par de restricciones disyuntivas con constante grande L y variable y_{ijhkl}); $Cmax \geq c_{j, h_j}$.

4.2 Extensión Rush Order

En $t^* = Cmax_0/2$ llega $J^R = \{r\}$; el conjunto pasa a $J = J^0 \cup J^R$. Del schedule inicial S^0 se conocen $s_{jh}^0, c_{jh}^0, x_{ijh}^0$. Las operaciones se clasifican en:

- **Terminadas** $F = \{(j, h): c_{jh}^0 \leq t^*\} \rightarrow$ congeladas.
- **En proceso** $I = \{(j, h): s_{jh}^0 < t^* < c_{jh}^0\} \rightarrow$ continúan sin interrupción.
- **Pendientes** $P = \{(j, h): s_{jh}^0 \geq t^*\} \rightarrow$ reprogramables.
- **Rush** $R = \{(r, h)\}$. Conjunto a reprogramar: $Q = P \cup R$.

Cada máquina queda disponible en $A_i^t = \max(t^*, \max_{(j, h) \in I: x_{ijh}^0 = 1} c_{jh}^0)$ (o t^* si no tiene operación en proceso). Las restricciones del modelo base se aplican sobre Q , agregando: $s_{jh} \geq t^*$; $s_{jh} \geq A_i^t - L(1 - x_{ijh})$; congelamiento $s_{jh} = s_{jh}^0, c_{jh} = c_{jh}^0, x_{ijh} = x_{ijh}^0 \forall (j, h) \in F \cup I$; y para la primera operación pendiente de cada trabajo, precedencia respecto del término congelado $c_{j, h-1}^0$.

4.3 Función objetivo extendida y nervousness

$$\min Z = Cmax_R + \alpha C_r + \beta N$$

donde $C_r = c_{r, h_r}$ es el término del rush order y la nervousness combina cambios de inicio y de máquina de las operaciones pendientes:

$$N = \underbrace{\gamma_1 \sum_{(j, h) \in P} |s_{jh} - s_{jh}^0|}_{N_s} + \underbrace{\gamma_2 \sum_{(j, h) \in P} (1 - x_{t^0 jh})}_{N_m}$$

El modelo se resuelve de forma metaheurística (GA); Z se usa como fitness en las variantes orientadas a urgencia/estabilidad y como criterio de etiquetado del selector inteligente.

Ejemplo numérico de Z (con los valores del caso de estudio, §6.2, y $\alpha=0.5, \beta=0.1$): mantener el programa intacto produce $Cmax_R=518, C_r=518, N=0$, luego $Z = 518 + 0.5 \cdot 518 + 0.1 \cdot 0 = 777$; reoptimizar todas las pendientes produce $Cmax_R=512, C_r=512,$

$N=747$, luego $Z = 512 + 256 + 74.7 = 842.7$. Los 6 segundos de makespan que gana la reoptimización cuestan 74.7 puntos de penalización por desorden: bajo esta función objetivo, la decisión correcta es no tocar el programa. Este trade-off es el objeto central del trabajo.

5. Metodología

5.1 Extracción de datos

Las Tablas 5 (máquinas factibles) y 6 (tiempos) del paper base se transcribieron a `data/datos_base_extraidos.csv` ($8 \times 5 = 40$ operaciones, 81 pares operación-máquina). El rush order canónico J9 (`data/datos_rush_order.csv`) es una reedición del modelo 8, consistente con el evento “nueva orden del octavo modelo” del paper.

5.2 Scheduling inicial (algoritmo genético)

El GA es la pieza técnica central de la metodología; esta sección lo desarma en sus componentes.

5.2.1 Representación: cromosoma de dos segmentos MS+OS

Una solución del FJSSP debe responder dos preguntas por cada operación: *¿en qué máquina?* y *¿en qué orden?* El cromosoma las separa en dos segmentos:

- **MS (Machine Selection):** un vector con una posición por operación (40 en el problema inicial) que indica la máquina elegida, siempre dentro del conjunto factible de la Tabla 5. Ejemplo: J1-O1 es factible en {M3 (38 s), M8 (49 s)}; su gen solo puede valer M3 o M8.
- **OS (Operation Sequence):** una lista de números de trabajo donde cada trabajo aparece tantas veces como operaciones tiene ($8 \times 5 = 40$ genes). La regla de lectura es la clave: **la k-ésima aparición del trabajo j representa su k-ésima operación.** Ejemplo con dos trabajos de dos operaciones: $OS = [2, 1, 1, 2]$ significa “considerar J2-O1, luego J1-O1, luego J1-O2, luego J2-O2”.

La propiedad que justifica esta codificación: **cualquier permutación de OS es válida.** Como las apariciones de j se leen en orden, es imposible expresar “O2 antes que O1” — la precedencia se cumple por construcción y el GA nunca genera ni repara soluciones infactibles.

5.2.2 Decodificación con inserción en huecos (schedules activos)

El decodificador recorre OS y coloca cada operación en el **primer hueco de su máquina donde quepa completa**, nunca antes del término de la operación anterior de su trabajo. Ejemplo: si la máquina está ocupada en $[10, 20]$ y $[30, 40]$, y una operación de duración 8 puede empezar desde $t = 12$, se coloca en $[20, 28]$ — dentro del hueco — en lugar de apilarse al final en $t = 40$. La distinción es relevante: apilar al final produce schedules *semi-activos*; rellenar huecos produce schedules *activos*, un subconjunto más pequeño del espacio de soluciones que contiene siempre al óptimo (Giffler & Thompson, 1960). Esta decisión explica parte de la diferencia con el paper base (§6.4).

El mismo decodificador acepta un estado inicial no vacío (máquinas bloqueadas hasta A_i^t , trabajos avanzados hasta su última operación congelada), de modo que **programar desde cero y reprogramar tras el rush son el mismo algoritmo con distinto estado de partida** — no hay dos implementaciones que puedan divergir.

5.2.3 Operadores evolutivos

- *Selección por torneo ($k=3$)*: para cada padre se sortean 3 individuos y gana el de mejor fitness. Presión selectiva moderada: los buenos se reproducen más, sin eliminar del todo la diversidad (evita convergencia prematura).
- *Cruce POX en OS (precedence-preserving order-based crossover)*: se sortea un subconjunto de trabajos; el hijo hereda las **posiciones exactas** de esos trabajos desde el padre 1 y rellena los espacios restantes con los demás trabajos **en el orden relativo** del padre 2. Ejemplo con padres [1,2,1,3,2,3] y [3,2,3,1,2,1], conservando {1}: el hijo fija los 1 en las posiciones 1 y 3, y rellena con la secuencia 3,2,3,2 del padre 2 $\rightarrow$ [1,3,1,2,3,2]. Cada trabajo conserva su número de apariciones, así que el hijo es siempre un OS válido sin reparación.
- *Cruce uniforme en MS*: cada gen de máquina se hereda de uno u otro padre con probabilidad 1/2 (ambos padres portan solo máquinas factibles, luego el hijo también).
- *Mutación ($p = 0.15$)*: reasignación de la máquina de una operación al azar (dentro de su conjunto factible) e intercambio de dos posiciones de OS. Aporta la diversidad que el cruce por sí solo no genera.
- *Elitismo (1)*: el mejor individuo pasa intacto a la siguiente generación — la calidad de la mejor solución nunca retrocede.

5.2.4 Presupuestos según el uso

El mismo motor corre con tres presupuestos, dimensionados por el costo pagable en cada contexto:

Uso	Población $\times$ generaciones	Corridas	Tiempo aprox.
Scheduling inicial	120 $\times$ 300	1	~ 4 s
Rescheduling (experimentos finales)	80 $\times$ 150	1	~ 1 s
Etiquetado del dataset del selector	30 $\times$ 40	3 (se toma la mediana)	~ 0.1 s c/u

5.2.5 Por qué GA y no un método exacto

El MILP del §4 necesita una variable binaria de orden y_{ijkl} por cada par de operaciones que comparten una máquina factible: para 40–45 operaciones son miles de binarias unidas por restricciones disyuntivas de constante grande — la estructura que peor escala en solvers exactos. El GA entrega soluciones de calidad en segundos, admite cambiar la función objetivo sin tocar el motor (requisito de las estrategias del §5.4, cada una con su propio fitness) y es el método de referencia del propio paper base (RLEGA es un GA mejorado con aprendizaje por refuerzo). La renuncia a la garantía de optimalidad resultó además auditable: en el subproblema del rush, el GA alcanza la cota inferior aritmética (§6.2), es decir, no dejó calidad sobre la mesa.

5.3 Simulación de la perturbación

En $t^* = C_{\max}/2$ el DT lógico (capa de monitoreo implementada como regla sobre el estado del sistema de pedidos) detecta la inserción de J9, clasifica las operaciones en F/I/P, congela FUI, calcula la disponibilidad A_i^t y construye el subproblema sobre $Q = P \cup R$.

5.4 Estrategias de rescheduling

1. **insert_end** (baseline sin recuperación): el programa pendiente queda intacto; el rush se agrega al final de cada máquina.
2. **right_shift** (inserción con desplazamiento): el rush se programa lo antes posible compitiendo solo con lo congelado, y las operaciones pendientes conservan su máquina y orden, desplazadas a la derecha únicamente lo necesario (nunca antes de su inicio original). Prioriza el término del rush a costa de retrasar el resto.
3. **partial_ga**: GA sobre Q minimizando $C_{\max_R}$.
4. **priority_ga**: GA sobre Q minimizando $C_{\max_R} + \alpha C_r$.
5. **stability_ga**: GA sobre Q minimizando $C_{\max_R} + \alpha C_r + \beta N$.

El menú recorre deliberadamente el espectro clásico del rescheduling (Vieira et al., 2003): las dos primeras son **reparaciones locales** (rápidas, estables, sin optimización) y las tres últimas son **regeneraciones parciales** (reoptimizan, con distinto balance entre desempeño, urgencia y estabilidad). Ninguna domina siempre a las demás — por eso tiene sentido un selector.

Las tres variantes con GA usan **warm start elitista**: el resultado reportado nunca es peor que las heurísticas deterministas evaluadas con el objetivo de la propia variante. La necesidad es empírica, no teórica: con el presupuesto reducido del etiquetado (población 30, 40 generaciones), un GA partiendo de población aleatoria obtiene para *stability_ga* valores Z entre 784 y 811 en el escenario canónico — *peor* que el 777 determinista de *insert_end*, al que en teoría domina. Sin warm start, ese déficit de convergencia contaminaría las etiquetas de entrenamiento del selector (escenarios etiquetados como “gana *insert_end*” que en realidad son empates); con él, la dominancia queda garantizada por construcción.

5.5 Selector inteligente (XGBoost)

5.5.1 Qué es XGBoost y por qué aquí

XGBoost (Chen & Guestrin, 2016) construye un conjunto de árboles de decisión en secuencia: cada árbol nuevo se entrena para corregir el error residual de los anteriores (*gradient boosting*), con regularización que limita el sobreajuste. Es el método de referencia para datos tabulares pequeños: no exige normalización ni ingeniería de features intensiva, tolera colinealidad, expone la importancia de cada variable y predice en microsegundos — compatible con decidir en línea dentro del ciclo del Digital Twin.

5.5.2 El rol del modelo: sustituto (surrogate) del experimento

Saber qué estrategia conviene exige, en principio, ejecutar las cinco y comparar — segundos de GA por estrategia. El selector aprende a **predecir el costo Z de cada estrategia sin ejecutarla**: la entrada es el vector de 14 features del estado del taller concatenado con la codificación one-hot de la estrategia (19 columnas en total; el dataset

de 300 escenarios $\times$ 5 estrategias produce 1500 filas de entrenamiento), y la salida es el Z predicho. Para decidir se predican los cinco costos y se toma el argmin. En el momento de la decisión el modelo no ejecuta ningún GA y su salida es **determinista**: toda la aleatoriedad del sistema está en la generación de los datos de entrenamiento, no en la decisión. Hiperparámetros: 300 árboles, profundidad 5, learning rate 0.08.

5.5.3 Las 14 variables predictoras

Feature	Definición
t_frac	$t^*/C_{\max 0}$: fracción del horizonte transcurrida al llegar el rush
n_terminadas, n_proceso, n_pendientes	Tamaños de los conjuntos F, I y P
carga_media, carga_max	Carga pendiente media y máxima por máquina (tiempo de proceso aún asignado)
holgura_media	Holgura promedio de las pendientes: media de $(s^0 - t^*)/C_{\max 0}$
rush_maquinas_factibles	Total de alternativas máquina-operación del rush (su flexibilidad)
rush_tiempo_min, rush_tiempo_medio	Duración de la cadena del rush sumando mínimos (cota de C_r) y promedios
saturation	Trabajo restante / capacidad disponible: $(\text{carga pendiente} + \text{rush mínimo}) / (m \cdot (C_{\max 0} - t^*))$
cmax0	Makespan del programa inicial
avance_tiempo_pct	Fracción del tiempo de trabajo total ya completada en t^*
pendientes_en_maquinas_rush	Operaciones pendientes cuya máquina original compite con las máquinas factibles del rush

5.5.4 Generación del dataset

300 escenarios sintéticos: rush aleatorio en cada iteración (3–5 operaciones; 1–3 máquinas factibles por operación; tiempos $U[30,105]$, el rango empírico de la Tabla 6), $t^* \sim U(0.3, 0.7) \cdot C_{\max 0}$ y cinco programas iniciales distintos (variación de la carga del taller). En cada escenario se ejecutan las 5 estrategias registrando los componentes ($C_{\max R}$, C_r , N); las tres con GA se etiquetan con la **mediana de 3 corridas** con semillas distintas, porque el ruido de una corrida única con presupuesto reducido — medido en 26–35 puntos de Z entre semillas — supera el margen típico entre estrategias. Distribución de ganadoras resultante: insert_end 33, right_shift 44, partial_ga 9, priority_ga 10, stability_ga 204 (68%).

5.5.5 Por qué regresión de costo y no clasificación

La formulación ingenua — un clasificador de “qué estrategia gana” — pierde la información de *por cuánto* gana, y con clases desbalanceadas (68% stability) eso es fatal. Considérense dos escenarios: en A, la mejor estrategia le gana a la estable por 1 punto de Z; en B, la estable gana por 60. Un clasificador con pesos balanceados, obligado a

atender las clases raras, aprende a desviarse en casos como A (gana 1 punto) al precio de equivocarse en casos como B (pierde 60): compra aciertos baratos pagando errores caros. La teoría de decisión sensible al costo (Elkan, 2001) prescribe la alternativa correcta: aprender el **costo esperado de cada acción** y elegir el argmin, que es exactamente la regresión adoptada. La diferencia empírica es contundente: regret medio 9.43 (clasificador balanceado) contra 1.18 (regresión de costo) — sección 6.3.

5.6 Diseño experimental

Cuatro escenarios obligatorios con semilla fija (reproducibles con python `src/run_experiments.py`): (1) scheduling inicial; (2) rush sin recuperación inteligente; (3) recuperación por GA parcial; (4) recuperación con selector XGBoost. Métricas: C_{max} , C_r , tiempo computacional, N, operaciones modificadas y % de recuperación

$$= \frac{C_{max_{pert}} - C_{max_{rec}}}{C_{max_{pert}} - C_{max_0}} \times 100.$$

Protocolo de validación del selector. Los 300 escenarios provienen de 5 programas iniciales base; un split aleatorio pondría escenarios del mismo programa en entrenamiento y test (features correlacionadas → fuga de información → métricas infladas). Se usa por eso **validación cruzada leave-one-group-out**: 5 folds, cada uno deja fuera *todos* los escenarios de un programa base, de modo que cada escenario recibe una predicción de un modelo que jamás vio su taller. Sobre esas predicciones out-of-fold se reportan dos familias de métricas: las de clasificación (accuracy y recall por clase, informativas pero secundarias) y las de **calidad de decisión**, que son las que importan: el Z medio obtenido al seguir cada política y su **regret** — cuánto peor que el oráculo (la mejor estrategia de cada escenario) quedó la política, en promedio. Toda política se compara contra dos referencias obligatorias: el **baseline trivial** “elegir siempre `stability_ga`” (la mejor política constante) y el **oráculo** (cota inferior inalcanzable). La comparación es pareada: todas las políticas se evalúan sobre los mismos 300 escenarios.

6. Resultados

6.1 Scheduling inicial

El GA obtiene $C_{max_0} = 382$ s en 4.9 s (Gantt en `results/gantt_inicial.png`). Referencias del paper base (Tabla 7, 5 corridas): RLEGA 397/400/404 (mín/prom/máx), GA 411/420/433, TS 435/453.6/466.

6.2 Perturbación y recuperación

Con $t^* = 191$ s, el estado del taller es: 18 operaciones terminadas, 7 en proceso, 15 pendientes; $Q = 15 + 5 = 20$ operaciones. La cadena mínima del rush J9 suma 321 s, de modo que **ninguna solución puede terminar el rush antes de $191 + 321 = 512$ s** (cota inferior).

Escenario	Estrategia	C_{max} (s)	C_r (s)	N	Ops. modif.	Tiempo (s)	% Recup.
1. Inicial	GA	382	—	—	—	3.3	—
2. Rush sin recuperación	insert_end	518	518	0	0	<0.1	0

Escenario	Estrategia	Cmax (s)	Cr (s)	N	Ops. modif.	Tiempo (s)	% Recup.
3. Recuperación GA parcial	partial_ga	512	512	747	12	0.6	4.4
4. Recuperación inteligente (comp.)	XGBoost → stability_ga	518	518	0	0	0.7	0
(comp.)	right_shift	583	512	541	4	<0.1	-47.8
(comp.)	priority_ga	512	512	693	13	0.7	4.4

Daño de la perturbación: $518 - 382 = 136$ s de makespan. El GA parcial alcanza **512 s, el óptimo demostrable** (coincide con la cota inferior), recuperando 6 s (4.4% del daño). Nótese right_shift: logra el término óptimo del rush ($Cr = 512$, la cota inferior) pero a costa de empujar el resto del programa hasta $Cmax = 583$ — un **% de recuperación negativo**, que ilustra que priorizar ciegamente al pedido urgente puede dañar más de lo que el propio rush dañó. Ganttts en results/gantt_perturbado.png, results/gantt_recuperado.png y results/gantt_recuperado_xgboost.png.

6.3 Decisión y validación del selector

Decisión en el escenario canónico. Evaluando $Z = Cmax_R + 0.5 \cdot C_r + 0.1 \cdot N$: insert_end/stability_ga obtienen $Z = 777$, partial_ga $Z = 842.7$ y priority_ga $Z = 837.3$. La estrategia elegida por XGBoost (**stability_ga**) es efectivamente la de menor Z : ganar 6 s de makespan costaría modificar 12 de las 15 operaciones pendientes, con 7 cambios de máquina ($N = 747$), y el modelo aprendió que ese trade-off no conviene.

Validación contra baselines. Para evitar fuga de información, la validación es **leave-one-group-out por schedule base**: los escenarios del programa inicial que se evalúa nunca se vieron en entrenamiento, y cada uno de los 300 escenarios recibe una predicción *out-of-fold* (results/evaluacion_selector.csv):

Política	Z medio	Regret medio vs oráculo
Oráculo (mejor estrategia por escenario)	740.2	0
Baseline trivial: siempre stability_ga	740.8	0.66
Selector (regresión de Z)	741.4	1.18
Clasificador XGBoost balanceado (descartado)	749.6	9.43

Lectura honesta: bajo validación estricta, el selector queda **levemente por debajo del baseline trivial** (+0.5 puntos de Z en promedio, consistente en los 5 folds) y **8 veces mejor que el clasificador**. La causa es estructural: el margen total que separa al baseline del oráculo es 0.66 puntos de Z (~0.1%) — en esta distribución de escenarios y con estos pesos, la política estable es prácticamente inmejorable y no hay señal suficiente para que ningún selector la supere.

¿Para qué sirve entonces el selector? El análisis de sensibilidad responde (results/sensibilidad_pesos.csv): recalculando las estrategias ganadoras con los mismos 300 escenarios bajo otros pesos:

α	β	insert_end	right_shift	partial_ga	priority_ga	stability_ga
0.5	0.1 (usado)	33	44	9	10	204
0.5	0.0	24	79	97	75	25
0.5	0.05	28	46	18	23	185
0.5	0.2	51	46	3	5	195
0.0	0.1	49	40	17	11	183
1.0	0.1	29	46	12	22	191

Si la estabilidad no importara ($\beta=0$), la política “siempre stability” ganaría solo 25/300 escenarios y sería un pésimo baseline; la dominancia de la estrategia estable **es una consecuencia del peso β , no una ley del taller**. La regla fija “elegir siempre stability” solo es buena mientras β sea alto; el selector, en cambio, se re-entrena con las nuevas etiquetas sin cambiar una línea de código. Ese es su valor: robustez ante cambios de política de la empresa, no ganar décimas de Z en la configuración actual.

La feature más importante del regresor es el **tiempo total mínimo del rush** (0.42), seguida de la carga media de máquinas (0.10) — coherente con el análisis: el costo Z está gobernado por la cadena crítica del propio rush, que fija la cota inferior de C_r . Con las features corregidas, las 14 aportan importancia no nula.

6.4 Comparación con el paper base

Protocolo de comparación. Comparar contra resultados publicados exige declarar qué es comparable y qué no. Entre este trabajo y Liu et al. (2022) difieren: el algoritmo (GA clásico vs RLEGA, un GA potenciado con aprendizaje por refuerzo), el esquema de decodificación (activo con inserción en huecos vs no reportado), los parámetros y el presupuesto de iteraciones, el número de corridas (una con semilla fija vs cinco del paper), el entorno de cómputo, y la función objetivo del rescheduling (Z extendida con urgencia y estabilidad vs makespan puro). En consecuencia, la comparación se limita a **coherencia de orden de magnitud** y en ningún punto se afirma superioridad de método:

- **Inicial:** nuestro GA (382) mejora el mejor RLEGA reportado (397). No se afirma réplica: el decodificador con inserción en huecos construye schedules activos (el paper no detalla el suyo), y difieren parámetros, número de corridas, semillas y entorno computacional.
- **Rescheduling:** el paper reporta $397 \rightarrow 521$ al insertar una nueva orden en $t = 200$ s; nuestro resultado $382 \rightarrow 518$ (sin recuperación) y $\rightarrow 512$ (GA parcial) es consistente en magnitud, con la misma estructura de evento ($t^* \approx 200$ s, nueva orden = octavo modelo).
- **Tiempos computacionales:** el paper no reporta tiempos del caso de estudio; los nuestros (≈ 1 s por rescheduling) son compatibles con decisión en línea.

7. Discusión

- **Calidad de solución.** El GA con decodificación por huecos es competitivo (382 vs 397 del RLEGA) y el rescheduling parcial alcanza el óptimo del subproblema (cota inferior 512), por lo que la calidad de recuperación no es mejorable con ningún otro método.

- **Impacto del rush order.** El daño (136 s) está dominado por la cadena crítica del propio rush (321 s lanzada en $t^* = 191$): el margen de recuperación vía re-secuenciamiento es estructuralmente pequeño en este escenario (máx. 6 s). Esto no es una debilidad del método sino una propiedad de la instancia: con rush orders más cortos o flexibles (como los del set de entrenamiento) las estrategias difieren mucho más.
- **Trade-off makespan–estabilidad.** El resultado central: recuperar 4.4% del makespan cuesta modificar 12 de las 15 operaciones pendientes, incluidos 7 cambios de máquina. Bajo la función Z , la mejor decisión es mantener el programa estable — exactamente lo que recomienda el selector XGBoost, en línea con el argumento de match-up de Moratori et al. (2010): no siempre conviene reoptimizar todo.
- **Rol del modelo inteligente.** *stability_ga* ganó en 204/300 escenarios de entrenamiento, pero en 96 ganaron otras estrategias: el repertorio no es redundante. La validación sin fuga (6.3) muestra que un clasificador de la clase ganadora decide claramente peor que el baseline trivial, y que la regresión de costo queda a 0.5 puntos de Z de él — la lección metodológica es que en selección de estrategias importa la *calidad de la decisión* (regret), no el accuracy, y que un baseline fuerte y un oráculo deben reportarse siempre. El análisis de sensibilidad demuestra que la dominancia de la política estable depende enteramente del peso β : el aporte del selector es adaptabilidad ante cambios de esa política, no décimas de Z en la configuración vigente.
- **Limitaciones.** Una sola instancia base ($8 \times 8 \times 5$); pesos α , β , γ fijados por juicio (aunque su efecto está cuantificado en la tabla de sensibilidad); margen baseline–oráculo mínimo (0.66 en Z) que hace inalcanzable superar al baseline en esta distribución; nervousness medida solo sobre operaciones pendientes; detección del evento modelada como regla del DT lógico (no hay planta física); GA sin garantía de optimalidad fuera de este subproblema (aquí verificable por la cota); las etiquetas usan la mediana de 3 corridas de GA, que reduce pero no elimina el ruido de etiquetado.

8. Conclusiones

Se implementó una metodología completa de scheduling dinámico para un FJSSP en contexto Digital Twin: scheduling inicial por GA ($C_{\max 0} = 382$ s, mejor que el RLEGA reportado), simulación del rush order en $t^* = C_{\max 0}/2$ con congelamiento de operaciones terminadas/en proceso, cinco estrategias de recuperación y un selector XGBoost (regresión de costo Z) entrenado con 300 escenarios sintéticos de rush aleatorio y validado contra el baseline trivial y el oráculo. El GA parcial alcanza el makespan recuperado óptimo (512 s) y el selector inteligente identifica correctamente la estrategia de menor impacto global (Z), privilegiando la estabilidad del programa cuando la ganancia de makespan es marginal. **El objetivo del trabajo se cumple:** el sistema detecta la perturbación, la absorbe sin interrumpir operaciones en curso y decide la recuperación con criterio cuantitativo.

Trabajo futuro: instancias múltiples y de mayor escala, calibración de α/β con el decisor, otras perturbaciones (falla de máquina, la segunda del paper base), selector entrenado sobre variantes parametrizadas continuas de estrategia, y conexión a un DT físico.

9. Referencias

- Liu, Z., Wang, Y., Liang, X., & Ma, Y. (2022). Digital twin-driven adaptive scheduling for flexible job shops. *Sustainability*, 14(9), 5340. <https://doi.org/10.3390/su14095340>
- Wang, J., Liu, Y., Ren, S., Wang, C., & Wang, W. (2022). A method for dynamic insertion order scheduling in flexible job shops based on digital twins. *Applied Sciences*, 12(23), 12430.
- Moratori, P., Petrovic, S., & Vázquez-Rodríguez, J. A. (2010). Integrating rush orders into existent schedules for a complex job shop problem. *Applied Intelligence*, 32, 205–215. <https://doi.org/10.1007/s10489-010-0215-6>

10. Anexos

A. Supuestos adoptados

1. **Pesos de la función Z:** $\alpha = 0.5$, $\beta = 0.1$, $\gamma_1 = 1$, $\gamma_2 = 10$ (un cambio de máquina ≈ 10 unidades de tiempo de perturbación). Fijados por juicio; el análisis de 6.3 muestra la decisión que inducen.
2. **Rush order canónico:** reedición del modelo 8, igual que el evento dinámico del paper base. Para el entrenamiento del selector, el rush es **aleatorio en cada iteración** (3–5 ops, 1–3 máquinas por op, tiempos $U[30,105]$ — rango empírico de la Tabla 6).
3. **Due date del rush:** no se define en el enunciado ni en el paper; se reporta C_r en su lugar y se omite la tardanza.
4. **t^* de entrenamiento:** $U(0.3, 0.7) \cdot C_{\max_0}$ para que el selector generalice alrededor del caso canónico $t^* = 0.5 \cdot C_{\max_0}$.
5. **Etiquetado del dataset:** mediana de 3 corridas de GA por estrategia (reduce el ruido de etiquetado); estrategias deterministas con una corrida. Los GA usan warm start elitista respecto de las heurísticas deterministas.
6. **Validación:** leave-one-group-out por schedule base (5 folds), para que ningún escenario de test comparta programa inicial con el entrenamiento.
7. **Informe PDF:** generado con pandoc (HTML + MathML) e impresión headless de Edge, sin LaTeX.

B. Datos y parámetros

- Datos del caso: data/datos_base_extraidos.csv (Tablas 5–6 del paper); rush: data/datos_rush_order.csv.
- GA inicial: población 120, 300 generaciones, torneo $k=3$, $p_{\text{mut}} = 0.15$, elitismo 1. GA de rescheduling (experimentos): población 80, 150 generaciones. GA de etiquetado del selector: población 30, 40 generaciones $\times$ 3 semillas (mediana); 300 escenarios $\times$ 5 estrategias en ≈ 160 s.
- Selector (regresor de costo): XGBRegressor, 300 árboles, profundidad 5, lr 0.08; entrada = 14 features + one-hot de estrategia (1500 filas de entrenamiento). Clasificador de comparación: XGBClassifier 200/4/0.1 con sample_weight balanceado.
- La transcripción de los datos del paper es verificable programáticamente contra el PDF con python src/verificar_datos.py (81/81 pares operación-máquina-tiempo idénticos).

C. Resultados y código

- Tabla completa: `results/tabla_resultados.csv`; importancia de features: `results/importancia_features.csv`; validación del selector: `results/evaluacion_selector.csv`; sensibilidad de pesos: `results/sensibilidad_pesos.csv`.
- Código fuente en `src/` (cada módulo incluye un autochequeo ejecutable de factibilidad: precedencias, no solapamiento, congelamiento y respeto de t^*).